

Question 1: (20 Marks)

Develop an application for managing event registrations for a fictional tech conference. The application should allow users to add new participants to the event and save the participant details to a file. Each participant entry should be saved in a structured format.

Requirements:

1. Participant Class:

- Create a Participant class with the following properties:
 - ParticipantID (int)
 - Name (string)
 - Email (string)

2. EventManager Class:

- Create an EventManager class with the following functionalities:
 - Add a new participant.

Save all participant details to a file named `registrations.txt` in the following format:

```
ParticipantID: 1, Name: Jane Doe, Email: jane.doe@example.com
ParticipantID: 2, Name: John Smith, Email: john.smith@example.com
```

3. Program Flow:

- The application should:
 - Prompt the user to enter details for a new participant (ParticipantID, Name, Email).
 - Add the new participant to a collection of participants.
 - Save the participant details to the file `registrations.txt` each time a new participant is added.

Tasks:

1. Implement the Participant class. (5)
2. Implement the EventManager class with methods to add a new participant and save participant details to the file. (10)
3. Implement the main program flow to interact with the user and utilize the EventManager class. (5)

```
using System;
using System.Collections.Generic;
using System.IO;
```

```
namespace EventRegistration
```

```
class Participant
{
    public int ParticipantID { get; set; }
}
```

```

public string Name { get; set; }
public string Email { get; set; set; }

```

```

}

class EventManager
{
    participants
    private List<Participant> = new List<Participant>();

    public void AddParticipant(Participant participant)
    {
        StreamWriter writer = new StreamWriter("registrations.txt", Append: true);
        writer.WriteLine(JsonParticipant = JsonSerializer.Serialize(participant));
        writer.WriteLine(JsonParticipant);
        writer.Close();
        Participants.Add(participant);
        SaveParticipantToFile(participant);
    }

    private void SaveParticipantToFile(Participant participant)
    {
        StreamWriter writer = new StreamWriter("registrations.txt", append: true);
        string JsonParticipant = JsonSerializer.Serialize(participant);
        writer.WriteLine(JsonParticipant);
        writer.Close();
    }
}

```

→ if need of separate storage from files

```

class Program
{
    static void Main(string[] args)
    {
        EventManager eventManager = new EventManager();

        while (true)
        {
            int ID;
            Console.WriteLine("Enter the Participant Id: ")
            ID = int.Parse(Console.ReadLine());

            string name;
            Console.WriteLine("Enter the Participant name: ");

```



```

name = Console.ReadLine();
Console.WriteLine("Enter Email: ");
string email;
email = Console.ReadLine();
Participant P = new Participant { ParticipantID = ID, Name = name, Email = email };
EventManager.AddParticipant(P);

```

```

Console.WriteLine("Participant added and saved to file.");
}
}
}

```

Question 2 (15 marks)

You are developing an ASP.NET Core MVC application for managing a shopping cart. The application allows users to add items to their cart and view the contents of the cart. Fill in the missing code in the following controller.

```

using identityFinalExamQuestion.Models;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System;
using System.Collections.Generic;
using System.Text.Json;

```

```

public class CartController : Controller

```

```

{
    [HttpGet]
    public IActionResult Index()
    {
        var cartItems = GetCartFromSession();
        return View(cartItems);
    }

```

```

    [HttpGet]
    public IActionResult AddToCart()
    {
        var cartItems = new List<CartItem>
        {
            new CartItem { Id = 1, Name = "Laptop", Price = 999.99m },
            new CartItem { Id = 2, Name = "Phone", Price = 499.99m }
        };

```

```

        return View(cartItems);
    }
}

```

```
}
```

[HttpPost] 1

```
public IActionResult AddToCart(List<CartItem> items)
```

```
{
```

```
    var cartItems = GetCartFromSession();
```

Add code here to add the items in to the cart (3)

```
    foreach (CartItem item in items)
    {
        cartItems.Add(item);
    }
```

→ To read prev cart (code is in save cart to session)

```
    // new cartItems list has all record
    // saving it to session
```

```
    SaveCartToSession(cartItems);
    return RedirectToAction("Index");
```

```
}
```

[HttpGet]

```
public IActionResult ClearCart() if (HttpContext.Session.Key.Contains("cartItems"))
```

```
{
```

Add code to clear the cart (1)

```
    { HttpContext.Session.Remove("cartItems");
```

```
    return RedirectToAction("Index");
```

```
}
```

```
private List<CartItem> GetCartFromSession()
```

```
{
```

Add code to get the data from the cart (5)

```
    List<CartItem> items = new new List<CartItem>();
```

```
    if (HttpContext.Session.Key.Contains("cartItems"))
```

```
    {
        string cart = HttpContext.Session.GetString("cartItems");
```

```
        items = JsonSerializer.Deserialize<List<CartItem>>(cart);
```

```
        return items;
```

```
    }
```

```
    else
```

```
    {
        return items;
```

```
}
```

```
private void SaveCartToSession(List<CartItem> cartItems)
```

```
{
```

Save cart data into session (5)

```
    if (HttpContext.Session.Key.Contains("cartItems")) OR (!HttpContext.Session.Key.Contains("cartItems"))
```

```
    {
        string jsonCart = JsonSerializer.Serialize(cartItems);
```

```
        HttpContext.Session.SetString("cartItems", jsonCart);
```

```
    }
```

```
}
```



```

else
{
    string alreadySavedCart = HttpContext.Session.GetString("cartItems");
    string JsonCart = JsonSerializer.Serialize(CartItems);
    string completeCart = alreadySavedCart + JsonCart;
    HttpContext.Session.SetString("cartItems", completeCart);
}
}

```

Question # 3 (10 marks)

Consider the following ASP.NET Core MVC Controller:

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Http;
using System;

namespace CookieExample.Controllers
{
    public class CookieController : Controller
    {
        public IActionResult Index()
        {
            // Set a cookie

            CookieOptions options = new CookieOptions
            {
                Expires = DateTime.Now.AddMinutes(5),
                HttpOnly = true
            };

```

```
Response.Cookies.Append("SessionID", "12345", options);
Response.Cookies.Append("UserName", "JohnDoe", options);

// Get the cookie values
string sessionId = Request.Cookies["SessionID"];
string userName = Request.Cookies["UserName"];

// Modify the UserName cookie
if (userName != null)
{
    userName = userName.ToUpper();
    Response.Cookies.Append("UserName", userName, options);
}

// Add a new cookie if it doesn't exist
if (Request.Cookies["VisitCount"] == null)
{
    Response.Cookies.Append("VisitCount", "1", options);
}
else
{
    int visitCount = int.Parse(Request.Cookies["VisitCount"]);
    visitCount++;
    Response.Cookies.Append("VisitCount", visitCount.ToString(), options);
}
```

```
// Remove the SessionID cookie
```

```
Response.Cookies.Delete("SessionID");
```

```
// Get the modified cookie values
```

```
string modifiedUserName = Request.Cookies["UserName"];
```

```
string visitCountValue = Request.Cookies["VisitCount"];
```

```
string sessionIdAfterDeletion = Request.Cookies["SessionID"];
```

```
// Return the cookie values to the view
```

```
ViewBag.SessionID = sessionId;
```

```
ViewBag.UserName = userName;
```

```
ViewBag.ModifiedUserName = modifiedUserName;
```

```
ViewBag.VisitCount = visitCountValue;
```

```
ViewBag.SessionIDAfterDeletion = sessionIdAfterDeletion;
```

```
return View();
```

```
}
```

```
}
```

```
}
```

What will be the value of `ViewBag.SessionID` when the Index action is executed?

12345

What will be the value of `ViewBag.UserName` when the Index action is executed?

JOHNDOE

What will be the value of `ViewBag.ModifiedUserName` when the Index action is executed?

JOHNDOE

What will be the value of ViewBag.VisitCount when the Index action is executed on the first visit?
What will it be on the second visit?

1 on first visit
2 on second visit

What will be the value of ViewBag.SessionIDAfterDeletion when the Index action is executed?
null

Question # 4 (10 marks)

Consider the following model, controller, and view and answer the given question.

Models/Order.cs

```
public class Order
{
    public int Id { get; set; }
    public string ProductName { get; set; }
    public int Quantity { get; set; }
    public decimal Price { get; set; }
    public decimal TotalAmount => Quantity * Price;
}
```

Controllers/OrderController.cs

```
public class OrderController : Controller
{
    [HttpGet]
    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Create(Order order)
    {
        // Simulate saving the order to the database
        TempData["SuccessMessage"] = $"Order for {order.Quantity} {order.ProductName}(s) placed successfully!";
        return RedirectToAction("Details", new { id = 1 });
    }

    [HttpGet]
    public IActionResult Details(int id)
```



```

{
    // Simulate retrieving the order from the database
    var order = new Order
    {
        Id = id,
        ProductName = "Laptop",
        Quantity = 2,
        Price = 999.99m
    };

    return View(order);
}
}

```

Views/Order/Create.cshtml:

```

@model Order
    asp-controller = "Order"
<form asp-action="Create" method="post">
    <div>
        <label asp-for="ProductName"></label>
        <input asp-for="ProductName" />
    </div>
    <div>
        <label asp-for="Quantity"></label>
        <input asp-for="Quantity" />
    </div>
    <div>
        <label asp-for="Price"></label>
        <input asp-for="Price" />
    </div>
    <button type="submit">Create</button>
</form>

```

Views/Order/Details.cshtml:

@model Order

<h2>Order Details</h2>

<p>Product Name: @Model.ProductName</p>

<p>Quantity: @Model.Quantity</p>

<p>Price: @Model.Price</p>

<p>Total Amount: @Model.TotalAmount</p>

@if (TempData["SuccessMessage"] != null)

{

<p>@TempData["SuccessMessage"]</p>

}

A user navigates to /Order/Create and fills out the form with the following values:

- ProductName: "Tablet"
- Quantity: 3
- Price: 300.00

The user submits the form.

What will the user see if the form is successfully submitted?

Order Details

Product Name: Laptop

Quantity: 2

Price: 999.98

Total Amount: 1999.98

Order for 3 Tablets placed successfully!

10

1999.98
1999.98